

고급머신러닝 보고서

심층신경망

Deep Neural Network

전자공학부 전자공학과
2020144005 김윤성
2024. 09. 24 제출

목차 INDEX

01 심층신경망이란

- 신경망
- 노드
- 퍼셉트론
- 은닉층
- 인공신경망
- 심층신경망

02 실습 #1)

- ANN 구현
- 코드 분석
- 실행 결과
- 보완할 점
- 해결 방법

03 실습 #2)

- DNN 구현
- 코드 분석
- 실행 결과

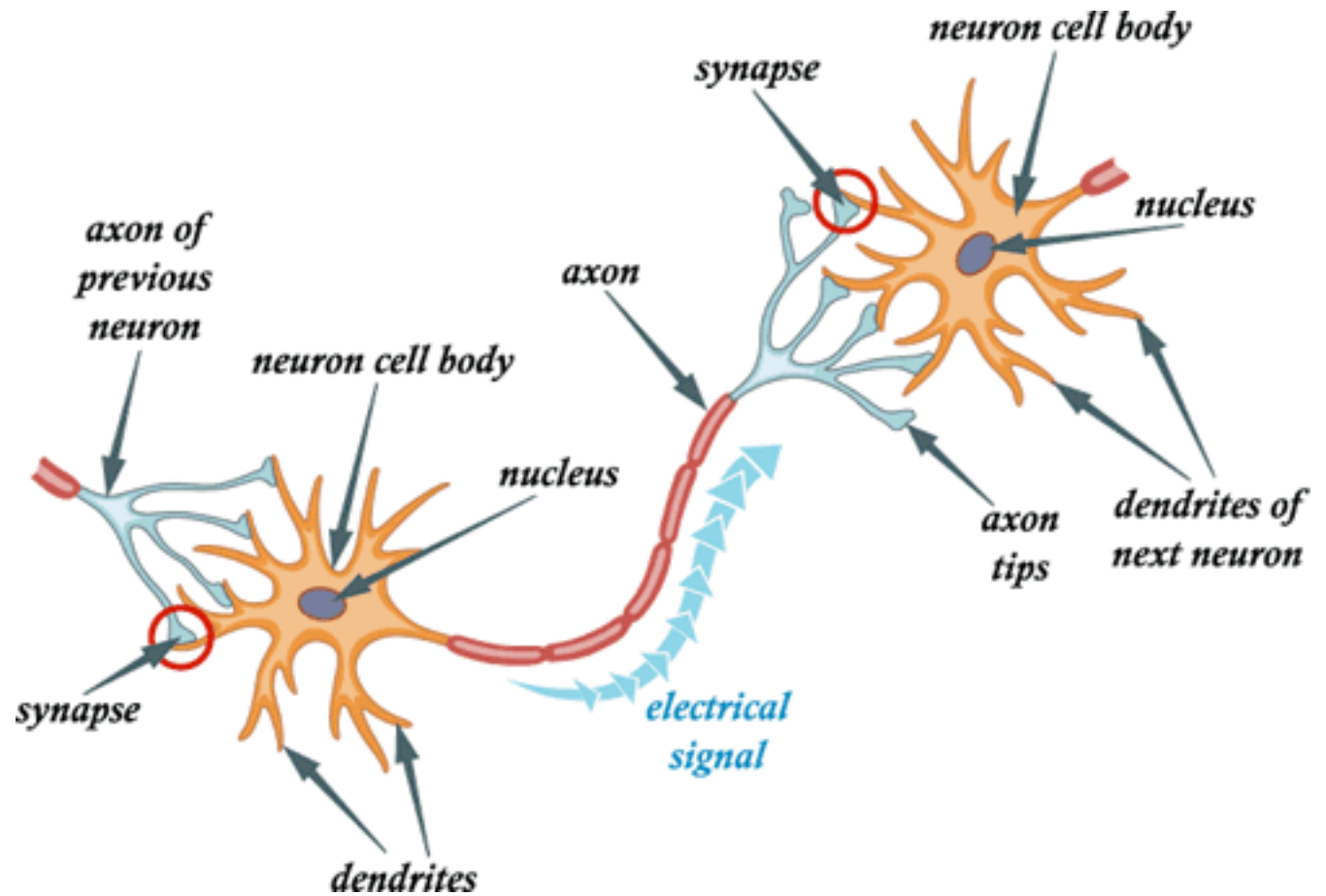
01

심층신경망이란

Deep Neural Network

신경망 Neural Network

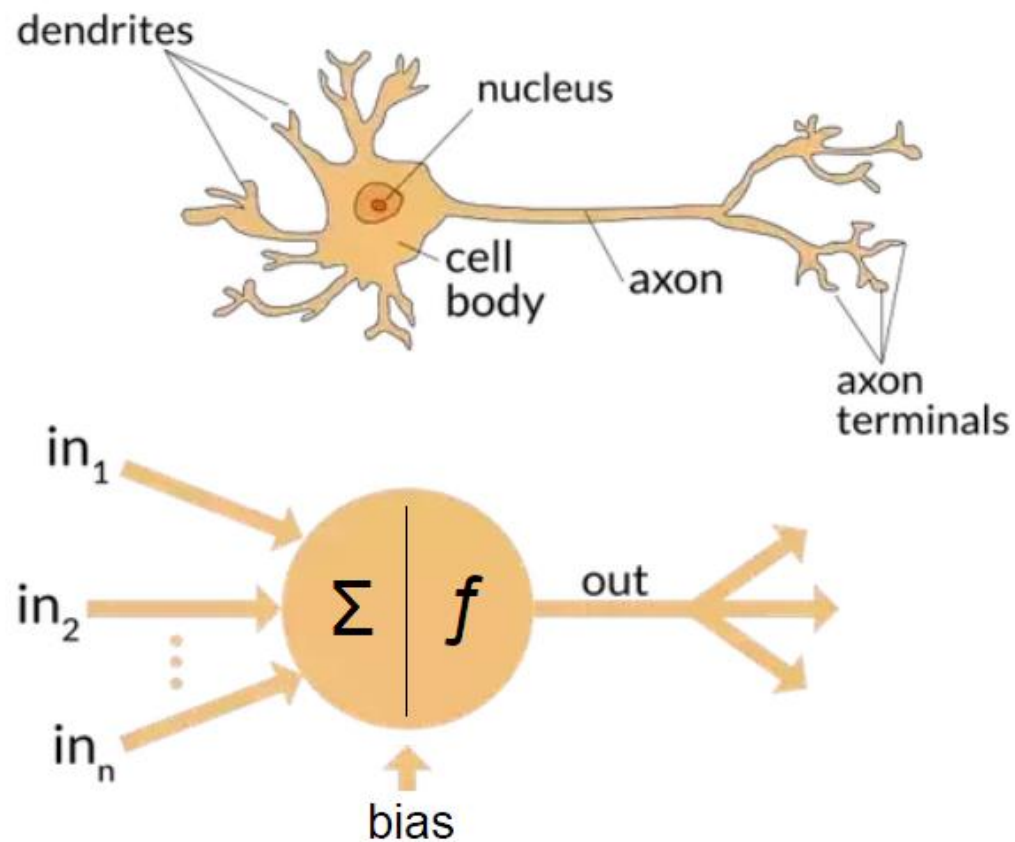
- 신경 세포(뉴런)의 집합체
- 전기적 신호로 다른 뉴런과 소통
- 시냅스에서 화학적인 신호로 변화
- 자극에 반응
- 학습, 기억 형성에 중요한 역할
- 신호의 세기에 따라 활성/비활성



01 심층신경망이란 노트

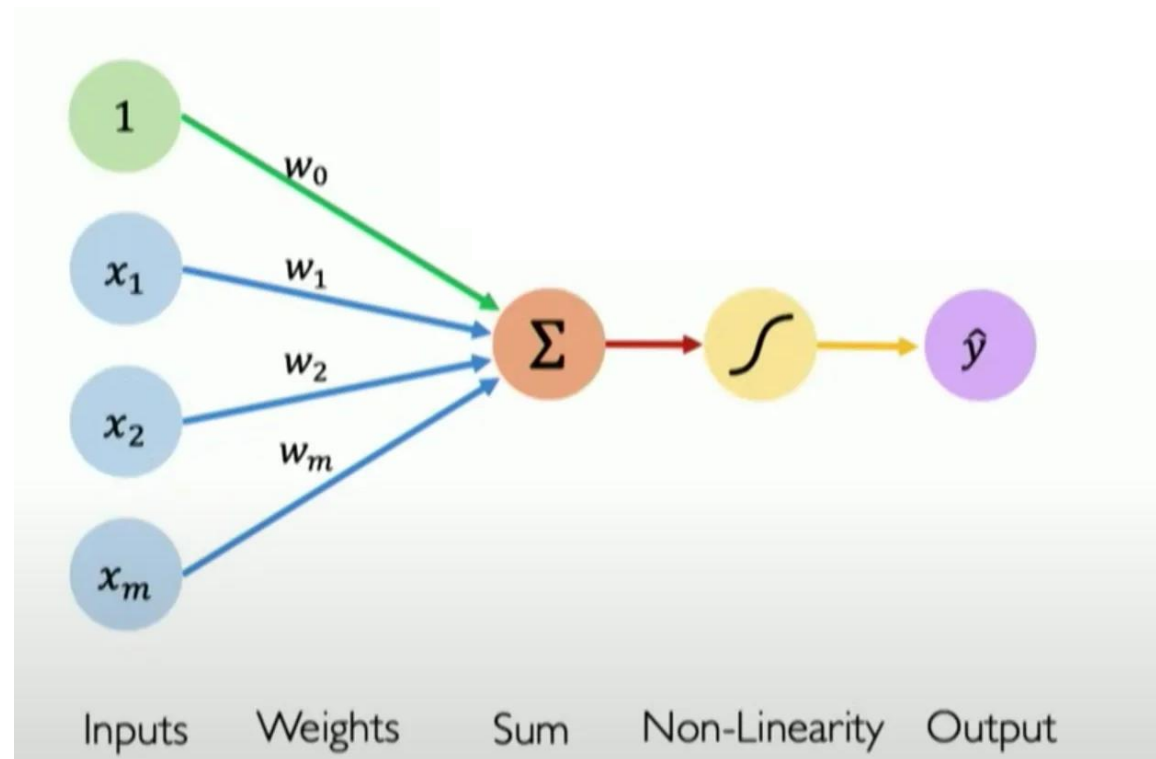
노드 Node

- 신경망에서 뉴런의 역할을 맡음
- 입력 데이터를 weight와 activation function을 통해 변환
- 신호의 세기에 따라 활성화/비활성
- layer 간의 연결을 통해 패턴 학습, 예측 수행
- 오류를 back propagation하여 가중치 조절
- 점점 더 정확한 예측을 하도록 최적화 -> 학습



퍼셉트론 Perceptron

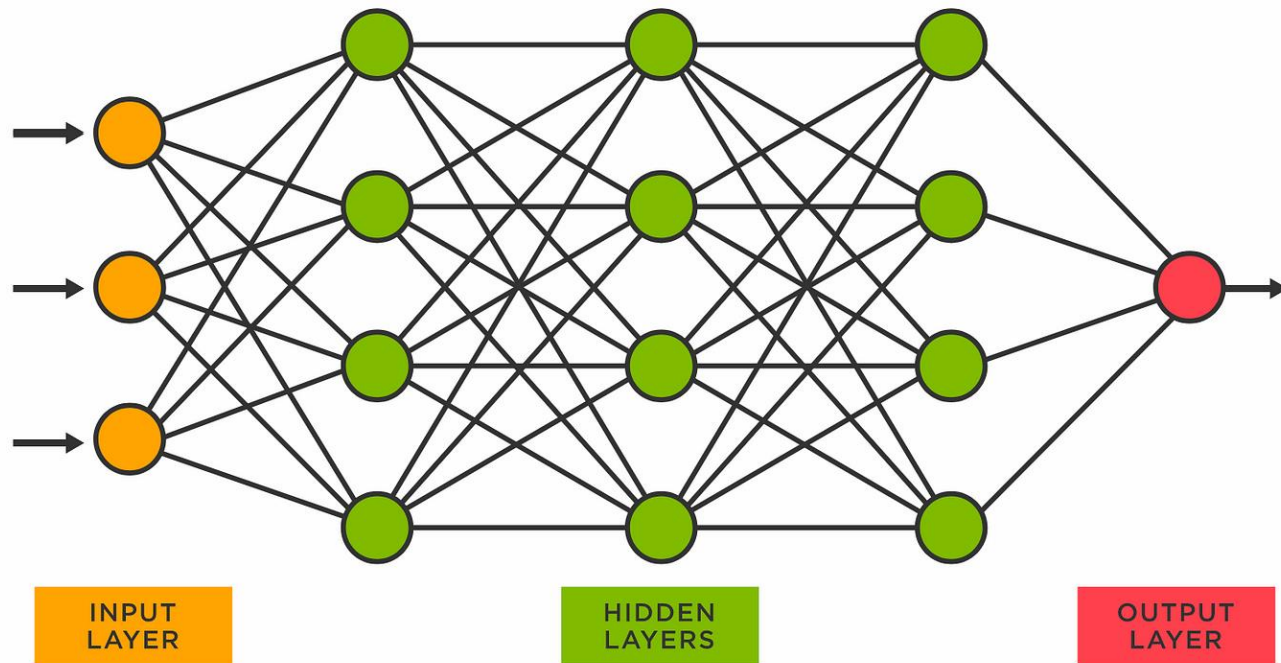
- 가장 간단한 인공신경망
- 가중치와 활성화 함수로 출력 값 생성
- 주로 이진 분류 문제(선형적인 문제) 해결에 사용
- 여러 퍼셉트론을 조합하면 비선형적 문제도 해결 가능



01 심층신경망이란 은닉층

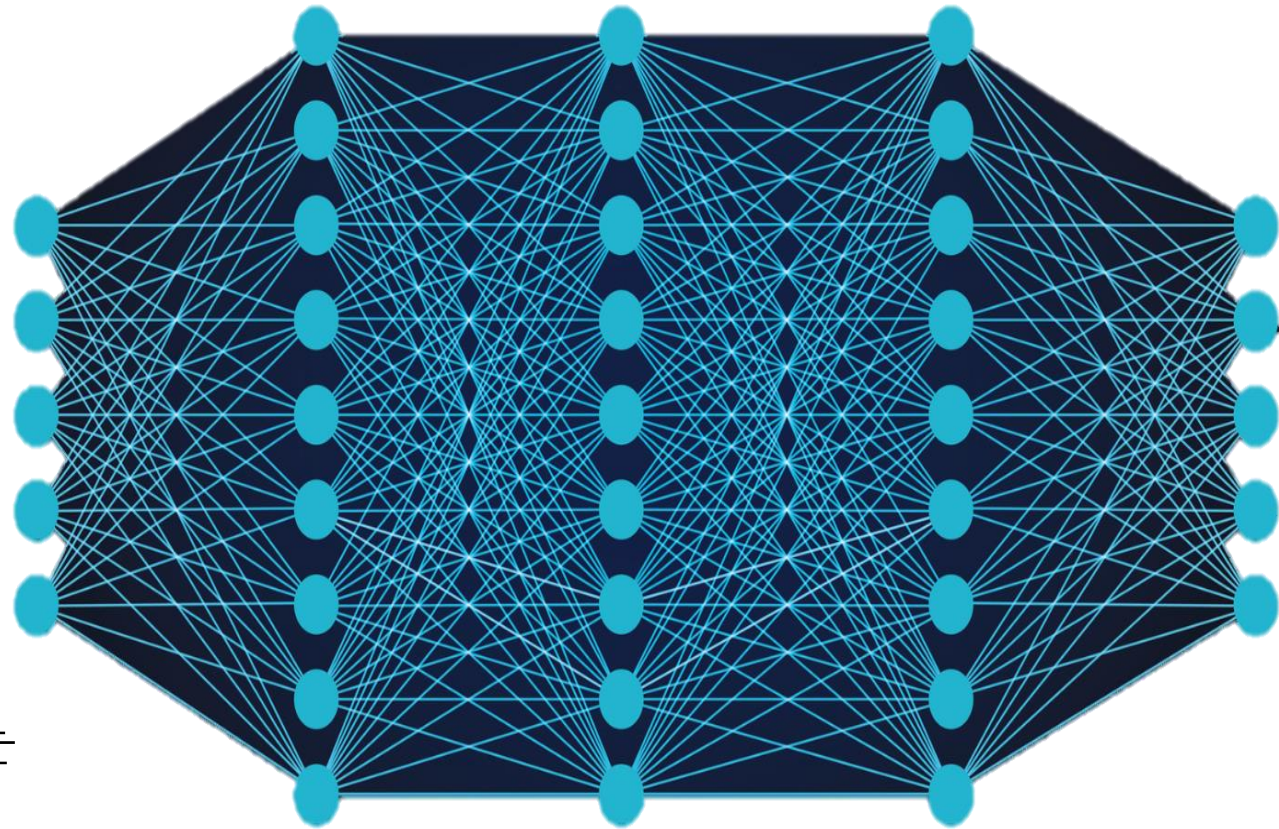
은닉층 Hidden Layers

- 입력과 출력 사이에 위치
- 신경망의 비선형 문제 해결을 가능하게 함
- 복잡한 관계 모델링, 더 높은 차원의 데이터 처리 가능
- 보다 정교하고 정확한 예측을 가능하게 함
- 은닉층의 수와 노드 수는 모델의 성능에 큰 영향



인공신경망 Artificial Neural Network

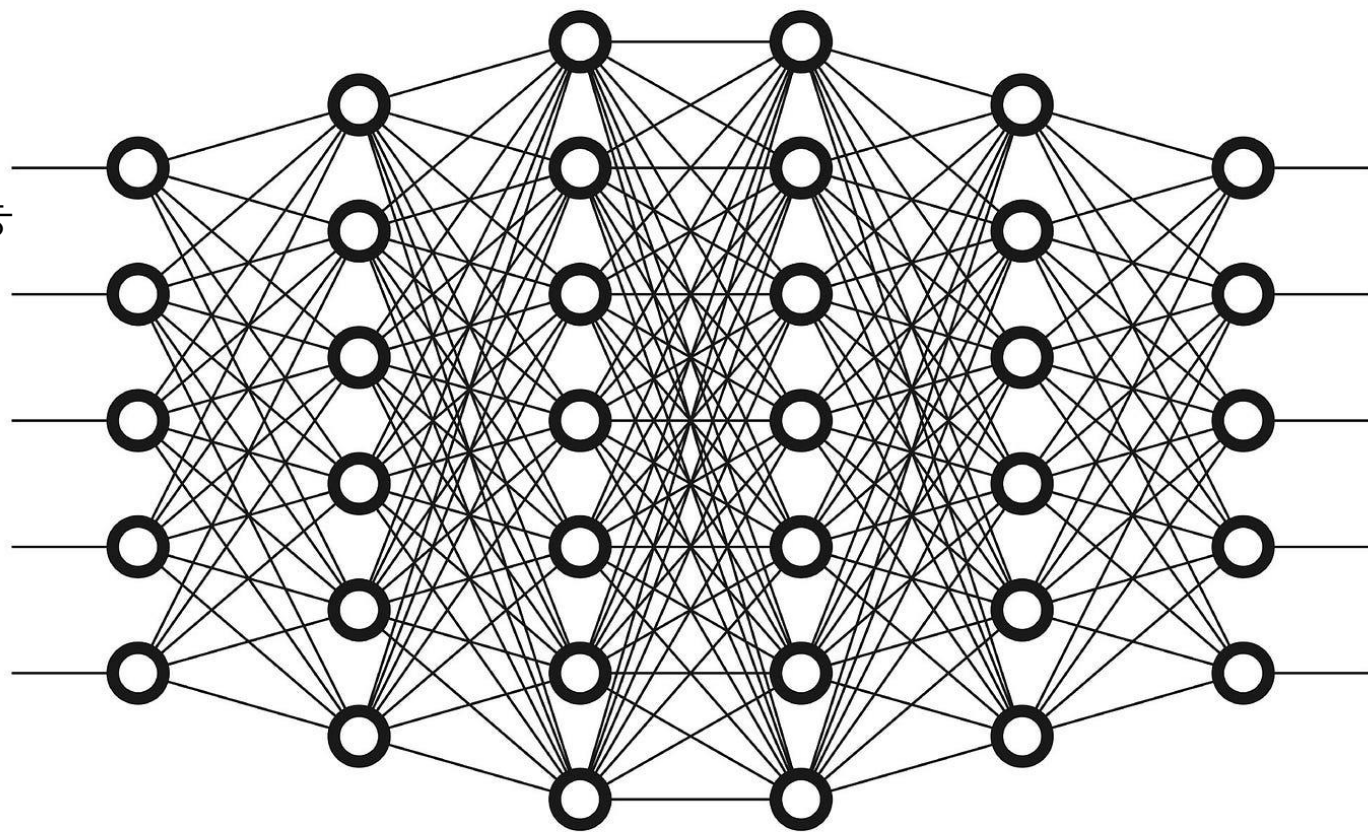
- 퍼셉트론이 모여 만들어진 신경망
 - 순전파와 역전파를 통해 스스로 오차 조정, 최적화
 - 비선형 관계 모델링 가능
 - 데이터를 기반으로 한 학습 능력
 - 새로운 데이터에 대한 예측이 가능
 - NLP, 이미지 인식 등 다양한 분야에서 활용 가능
- ▶ DNN과 CNN 등도 ANN의 일종이나, 본 보고서에서는 Two-Layer Neural Network를 ANN이라 지칭함



심층신경망

Deep Neural Network

- 인공신경망에서 은닉층을 늘린 형태
- 인공신경망보다 더 고차원의 데이터 학습 및 처리 가능
- 인공신경망 대비 비선형성 강화
- 입력 값에서 특징 추출 가능
- 대량의 데이터 처리에 유리



02

실습 #1

Two-Layer Neural Network 구현

실습 #1) Two-Layer Neural Network 구현

- 주어진 데이터의 y값을 One-Hot Encoding
- 은닉층의 노드는 5개로 고정
- 모든 활성화 함수는 Sigmoid 사용
- Weight 업데이트 시, 반복문(for, while 등)을 사용하지 않고 업데이트
- 데이터 개수에 따른 반복문은 사용 O

02 실습 #1 코드 분석

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

필요한 모듈 임포트

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_d(x):
    return x * (1 - x)

def One_Hot_Encoding(y, number_of_class):
    one_hot = np.zeros((len(y), number_of_class))
    if (min(y) == 0):
        for i in range(len(y)):
            one_hot[i, int(y[i])] = 1
    else:
        for i in range(len(y)):
            one_hot[i, int(y[i] - 1)] = 1
    return one_hot
```

필요한 함수들을 정의한다.

sigmoid() : 활성화 함수값 반환

sigmoid_d() : 활성화 함수의 미분값 반환

One_Hot_Encoding() : y의 개수에 따라 자동으로 one hot encoding 시행

02 실습 #1 코드 분석

```
folder_location = "C:\\YS\\TUK\\Python_Class\\Advanced_Machine_Learning\\source\\NN_data.csv"
data = pd.read_csv(folder_location).drop(columns=['Unnamed: 0'])
X = data.iloc[:, :-1].values # 입력 데이터 (x0, x1, x2)
y = data.iloc[:, -1].values # 출력 데이터 (클래스)
num_classes = len(np.unique(y)) # 클래스 수
y_one_hot = One_Hot_Encoding(y, num_classes)
Loss_history = []
Accuracy_history = []
```

NN_data.csv를 불러오고, 전처리 시행
이 때, y값은 one-hot encoding해 둔다.

```
input_size = X.shape[1]
hidden_size = 5 # 히든 레이어의 노드 수
output_size = len(np.unique(y)) # 출력 클래스 수
learning_rate = 0.1
epochs = 10000
```

Hyper Parameter들을 정의

```
np.random.seed(42)
W1 = np.random.randn(input_size, hidden_size) # 입력 -> 히든 weight
W2 = np.random.randn(hidden_size, output_size) # 히든 -> 출력 weight
```

np.random의 seed()를 이용하여 가중치를 랜덤으로 초기화시킨다.

02 실습 #1 코드 분석

```
for epoch in range(epochs):
    hidden_input = np.dot(X, W1) # 입력 -> 히든
    hidden_output = sigmoid(hidden_input) # Activation Function (Sigmoid) 적용
    final_input = np.dot(hidden_output, W2) # 히든 -> 출력
    final_output = sigmoid(final_input) # Activation Function (Sigmoid) 적용
```

설정한 반복횟수(epoch)만큼 반복하기 위해 for 문 사용

입력에서 출력까지의 과정: $X \times W1 \rightarrow \text{Sigmoid} \times W2 \rightarrow \text{Sigmoid} \rightarrow Y$

hidden_input: $(900, 3) \times (3, 5) = (900, 5)$ **final_input:** $(900, 5) \times (5, 3) = (900, 3)$

```
error = y_one_hot - final_output
if epoch % 1000 == 0:
    print(f'Epoch {epoch}, Loss: {np.mean(np.abs(error))}')
if epoch % 10 == 0:
    Loss_history.append(np.mean(np.abs(error)))
    y_pred = np.zeros_like(final_output)
    y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
    accuracy = np.mean(np.all(y_pred == y_one_hot, axis=1))
    Accuracy_history.append(accuracy)
```

epoch 1000번마다 Loss를 출력하고, epoch 10번마다 Loss 와 Accuracy를 기록

```
dif_output = error * sigmoid_d(final_output)
dif_hidden = np.dot(dif_output, W2.T) * sigmoid_d(hidden_output)
W2 += np.dot(hidden_output.T, dif_output) * learning_rate # 히든 -> 출력 weight update
W1 += np.dot(X.T, dif_hidden) * learning_rate # 입력 -> 히든 weight update
```

연산된 값으로 가중치를 업데이트하는 역전파과정
출력에 가까운 가중치부터 차례대로 업데이트
배열을 이용하여 반복문을 중첩시키지 않고 한 번에 업데이트

02 실습 #1 코드 분석

```
hidden_input = np.dot(X, W1)
hidden_output = sigmoid(hidden_input)
final_input = np.dot(hidden_output, W2)
final_output = sigmoid(final_input)

y_pred = np.zeros_like(final_output)
y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
accuracy = np.mean(np.all(y_pred == y_one_hot, axis=1))
print(f'Accuracy: {accuracy * 100:.2f}%')
```

예측값과 최종 정확도를 계산하는 부분이다.

Forward Propagation을 한 후 해당 출력값을 One-Hot Encoding 한 후, 실제 값과 비교하여 정확도를 계산한다.

02 실습 #1 코드 분석

```
epochs_range = range(0, epochs, 10)
fig, ax1 = plt.subplots()

ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
loss_plot, = ax1.plot(epochs_range, Loss_history, color='tab:red', label='Loss')
ax1.tick_params(axis='y', labelcolor='tab:red')

ax2 = ax1.twinx()
ax2.set_ylabel('Accuracy [%]')
accuracy_plot, = ax2.plot(epochs_range, Accuracy_history, color='tab:blue', label='Accuracy')
ax2.tick_params(axis='y', labelcolor='tab:blue')

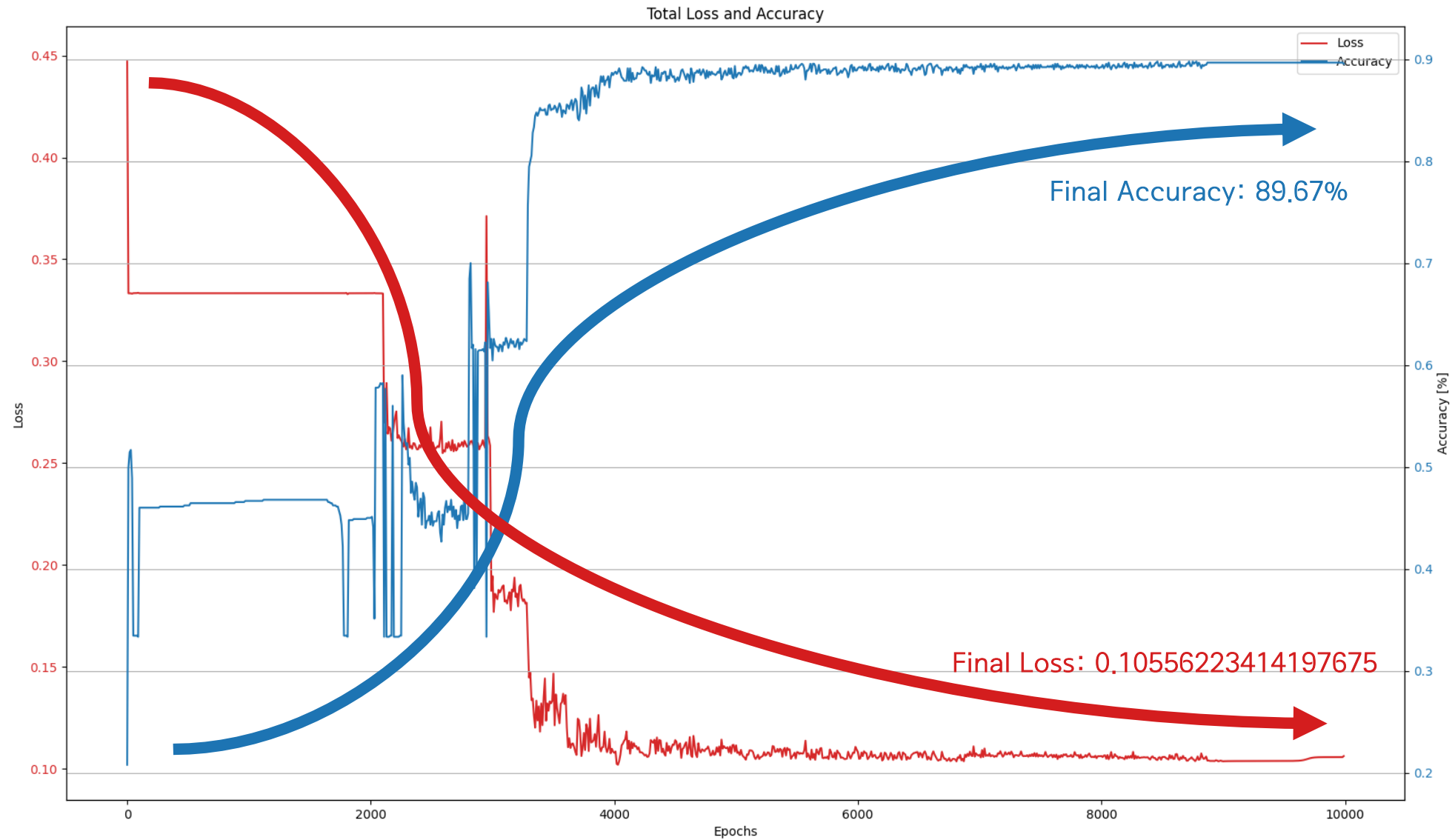
fig.tight_layout()
plt.title('Total Loss and Accuracy')
plt.grid(True)
lines = [loss_plot, accuracy_plot]
labels = [line.get_label() for line in lines]
ax1.legend(lines, labels, loc=0)

plt.show()
```

그래프를 그리는 부분이다.

정확도와 Loss를 그래프로 나타내는데, X축을 공유하여 한 평면 안에 두 개의 그래프를 나타낼 수 있게 하였다.

02 실습 #1 실행 결과



인공신경망

Artificial Neural Network

- Hidden_Layer가 1개여서 심화된 학습이 불가능
- 비선형 문제 해결 능력이 떨어짐
- 복잡한 관계 모델링, 더 높은 차원의 데이터 처리 기능 필요
- 보다 정교하고 정확한 예측 불가

=> Hidden Layer가 부족하기 때문

심층신경망

Deep Neural Network

- Hidden Layer의 개수를 늘림
- 비선형 문제 해결 능력 강화
- 복잡한 관계 모델링, 더 높은 차원의 데이터 처리 가능
- 보다 정교하고 정확한 예측 가능

03

실습 #2

Deep Neural Network 구현

실습 #2) Deep Neural Network 구현

- 실습 #1)에서 Hidden Layer의 개수를 1개에서 2개로 증가
- Hidden Layer의 Node 수는 5개로 고정
- $\hat{y}$ (예측값)에서 0.5를 기준으로 0 or 1로 변환
- 전체 Training set에 대해서 $\hat{y} = y$ 인 데이터 개수 count => 정확도 계산

03 실습 #2 코드 분석

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

필요한 모듈 임포트

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_d(x):
    return x * (1 - x)

def One_Hot_Encoding(y, number_of_class):
    one_hot = np.zeros((len(y), number_of_class))
    if (min(y) == 0):
        for i in range(len(y)):
            one_hot[i, int(y[i])] = 1
    else:
        for i in range(len(y)):
            one_hot[i, int(y[i] - 1)] = 1
    return one_hot
```

필요한 함수들을 정의한다.

sigmoid() : 활성화 함수값 반환

sigmoid_d() : 활성화 함수의 미분값 반환

One_Hot_Encoding() : y의 개수에 따라 자동으로 one hot encoding 시행

03 실습 #2 코드 분석

```
num_hidden_layers = int(input("Hidden Layer의 개수 입력: "))
hidden_layer_sizes = []
for I in range(num_hidden_layers):
    size = int(input(f" {i + 1}번째 Hidden Layer의 노드 수 입력: "))
    hidden_layer_sizes.append(size)
```

Hidden Layer와 각각의 Hidden Layer의 Node 수를 입력받는다.
그 값들은 hidden_layer_sizes에 저장한다.

```
folder_location = "C:\\YS\\TUK\\Python_Class\\Advanced_Machine_Learning\\source\\NN_data.csv"
data = pd.read_csv(folder_location).drop(columns=['Unnamed: 0'])
X = data.iloc[:, :-1].values # 입력 데이터 (x0, x1, x2)
y = data.iloc[:, -1].values # 출력 데이터 (클래스)
num_classes = len(np.unique(y)) # 클래스 수
y_one_hot = One_Hot_Encoding(y, num_classes)
Loss_history = []
Accuracy_history = []
```

NN_data.csv를 불러오고, 전처리 시행
이 때, y값은 one-hot encoding해 둔다.

03 실습 #2 코드 분석

```
input_size = X.shape[1] # 입력 속성 수
output_size = num_classes # 출력 클래스 수
learning_rate = 0.1 # 학습률
epochs = 10000 # 학습 반복 수
graph_scale = 10 # 그래프의 점을 얼마나 촘촘하게 찍을것인지, int (숫자가 작아질수록 촘촘함)
```

Hyper Parameter들을 정의한다.

이 때, graph_scale은 몇 epoch마다 정확도와 Loss를 기록할 지 제어하는 변수이다.

```
np.random.seed(42)
weights = []
weights.append(np.random.randn(input_size, hidden_layer_sizes[0])) # 입력 -> 첫 번째 히든레이어의 가중치
for i in range(1, num_hidden_layers):
    weights.append(np.random.randn(hidden_layer_sizes[i - 1], hidden_layer_sizes[i])) # 각 히든레이어 사이의 가중치
weights.append(np.random.randn(hidden_layer_sizes[-1], output_size)) # 히든레이어 -> 출력 사이의 가중치
```

실습 #1)과 비슷하게 np.random의 seed()를 이용하여 가중치를 초기화하나, 이번에는 가중치 배열이 2개가 아니기에 반복문을 이용하여 모든 가중치 배열의 값을 랜덤하게 초기화시킨다.

03 실습 #2 코드 분석

```
for epoch in range(epochs):
    # Forward Propagation
    layer_inputs = [X]
    layer_outputs = []
    # Forward Propagation during Hidden Layers
    for i in range(num_hidden_layers):
        input_to_layer = np.dot(layer_inputs[-1], weights[i])
        output_from_layer = sigmoid(input_to_layer)
        layer_inputs.append(input_to_layer)
        layer_outputs.append(output_from_layer)
    # 출력 레이어로 전달
    final_input = np.dot(layer_outputs[-1], weights[-1])
    final_output = sigmoid(final_input)
```

Propagation 부분의 Forward Propagation 부분
Hidden Layer가 1개가 아니기에 반복문을 사용하여 각 Layer들을 연산한다.
이렇게 Hidden Layer를 지나온 값들은 Sigmoid를 통과하여 출력 레이어로 전달된다.

```
error = y_one_hot - final_output
if epoch % 1000 == 0:
    print(f'Epoch {epoch}, Loss: {np.mean(np.abs(error))}')
if epoch % graph_scale == 0:
    Loss_history.append(np.mean(np.abs(error)))
    y_pred = np.zeros_like(final_output)
    y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
    accuracy = np.mean(np.all(y_pred == y_one_hot, axis=1))
    Accuracy_history.append(accuracy)
```

오차와 그래프를 위한 Loss, 정확도를 구하는 부분
아까 정의했던 graph_scale이 쓰이는 부분이다.

03 실습 #2 코드 분석

```
d_output = error * sigmoid_d(final_output)
d_hidden = [d_output]
for i in range(num_hidden_layers - 1, -1, -1): # for(int i=num_...; i <= 0; i--) 이라는 의미
    d_hidden_layer = np.dot(d_hidden[0], weights[i + 1].T) * sigmoid_d(layer_outputs[i])
    d_hidden.insert(0, d_hidden_layer) # 각 히든레이어들의 미분값 계산
weights[-1] += np.dot(layer_outputs[-1].T, d_output) * learning_rate # 마지막 히든 -> 출력 사이의 가중치 업데이트
for i in range(num_hidden_layers - 1, 0, -1):
    weights[i] += np.dot(layer_outputs[i - 1].T, d_hidden[i]) * learning_rate # 각 히든레이어 사이의 가중치 업데이트
weights[0] += np.dot(X.T, d_hidden[0]) * learning_rate # 입력 -> 히든 사이의 가중치 업데이트
```

Propagation 부분의 Backward Propagation 부분

Hidden Layer가 1개가 아니기에 반복문을 사용하여 각 Layer들을 연산한다.

weights에 정리된 Hidden Layer의 개수만큼의 가중치 배열들을 반복문을 이용하여 업데이트

```
layer_inputs = [X]
layer_outputs = []

for i in range(num_hidden_layers):
    input_to_layer = np.dot(layer_inputs[-1], weights[i])
    output_from_layer = sigmoid(input_to_layer)
    layer_inputs.append(input_to_layer)
    layer_outputs.append(output_from_layer)
final_input = np.dot(layer_outputs[-1], weights[-1])
final_output = sigmoid(final_input)
# 예측값 one hot encoding, 정확도 계산
y_pred = np.zeros_like(final_output)
y_pred[np.arange(len(final_output)), final_output.argmax(1)] = 1
accuracy = np.mean(np.all(y_pred == y_one_hot, axis=1))
print(f'Accuracy: {accuracy * 100:.2f}%')
```

학습 후 정확도를 구하기 위해 예측값을 one-hot encoding하여 정확도를 구하는 부분

03 실습 #2 코드 분석

```
epochs_range = range(0, epochs, graph_scale)
fig, ax1 = plt.subplots()

ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
loss_plot, = ax1.plot(epochs_range, Loss_history, color='tab:red', label='Loss')
ax1.tick_params(axis='y', labelcolor='tab:red')

ax2 = ax1.twinx()
ax2.set_ylabel('Accuracy [%]')
accuracy_plot, = ax2.plot(epochs_range, Accuracy_history, color='tab:blue', label='Accuracy')
ax2.tick_params(axis='y', labelcolor='tab:blue')

fig.tight_layout()
plt.title('Total Loss and Accuracy')
plt.grid(True)
lines = [loss_plot, accuracy_plot]
labels = [line.get_label() for line in lines]
ax1.legend(lines, labels, loc=0)

plt.show()
```

그래프를 그리는 부분이다.

정확도와 Loss를 그래프로 나타내는데, X축을 공유하여 한 평면 안에 두 개의 그래프를 나타낼 수 있게 하였다.

03 실습 #2 실행 결과

```
Epoch 0, Loss: 0.44709397856601846
Epoch 1000, Loss: 0.33333312243234914
Epoch 2000, Loss: 0.33334708106296324
Epoch 3000, Loss: 0.1945049687237915
Epoch 4000, Loss: 0.10780429734296917
Epoch 5000, Loss: 0.1114681594641887
Epoch 6000, Loss: 0.10823343548675708
Epoch 7000, Loss: 0.106866225273889
Epoch 8000, Loss: 0.10621649154494514
Epoch 9000, Loss: 0.10362828568417717
Accuracy: 89.67%
```

```
Hidden Layer의 개수 입력: 2
1번째 Hidden Layer의 노드 수 입력: 5
2번째 Hidden Layer의 노드 수 입력: 5
Epoch 0, Loss: 0.5564910494166503
Epoch 1000, Loss: 0.17769705587501336
Epoch 2000, Loss: 0.11256404811163537
Epoch 3000, Loss: 0.10217559437649937
Epoch 4000, Loss: 0.10535819757782929
Epoch 5000, Loss: 0.10371237743428832
Epoch 6000, Loss: 0.10681502874573326
Epoch 7000, Loss: 0.10010125482035989
Epoch 8000, Loss: 0.09990422622958554
Epoch 9000, Loss: 0.09977584143130626
Accuracy: 88.44%
```

ANN (히든노드 총 5개)

- 정확도: 89.67%
- 최종 오차: 0.106

DNN (히든노드 총 10개, 5 - 5)

- 정확도: 88.44%
- 최종 오차: 0.108

03 실습 #2 실행 결과

Hidden Layer의 개수 입력: 3
1번째 Hidden Layer의 노드 수 입력: 5
2번째 Hidden Layer의 노드 수 입력: 5
3번째 Hidden Layer의 노드 수 입력: 5
Epoch 0, Loss: 0.5791357936939185
Epoch 1000, Loss: 0.11092793106017024
Epoch 2000, Loss: 0.11130744758697544
Epoch 3000, Loss: 0.11535660156747894
Epoch 4000, Loss: 0.1228796696337631
Epoch 5000, Loss: 0.1252662041374865
Epoch 6000, Loss: 0.12279155645504411
Epoch 7000, Loss: 0.11355005418857832
Epoch 8000, Loss: 0.10222854372779266
Epoch 9000, Loss: 0.10214891289849679
Accuracy: 90.44%

Hidden Layer의 개수 입력: 3
1번째 Hidden Layer의 노드 수 입력: 7
2번째 Hidden Layer의 노드 수 입력: 8
3번째 Hidden Layer의 노드 수 입력: 10
Epoch 0, Loss: 0.6122107655347906
Epoch 1000, Loss: 0.09598160940957805
Epoch 2000, Loss: 0.10115597374478542
Epoch 3000, Loss: 0.10008868510060034
Epoch 4000, Loss: 0.09854450017491856
Epoch 5000, Loss: 0.09746902958098085
Epoch 6000, Loss: 0.08791721448020363
Epoch 7000, Loss: 0.09073448140481992
Epoch 8000, Loss: 0.09321085327128811
Epoch 9000, Loss: 0.08481028711643464
Accuracy: 90.56%

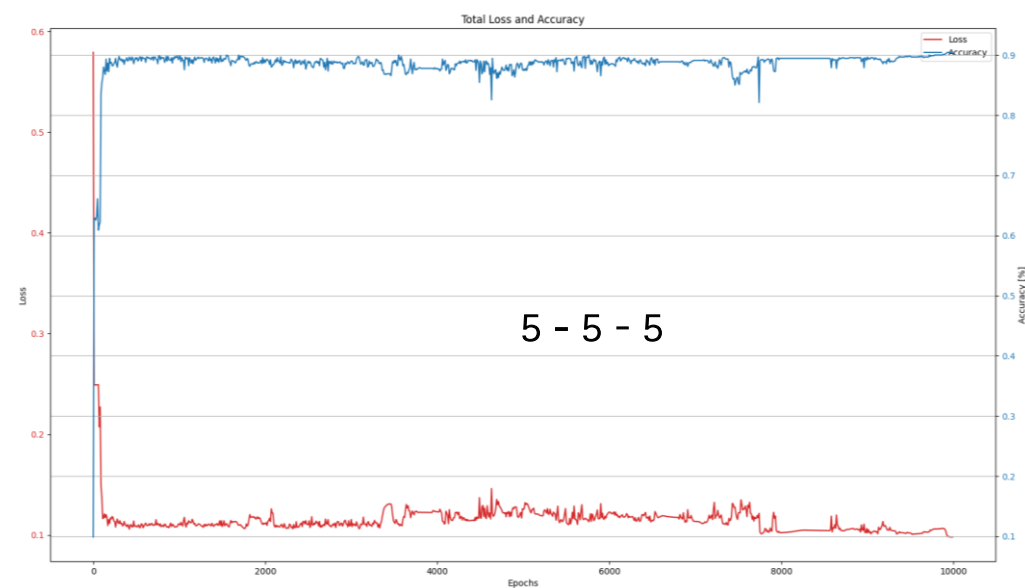
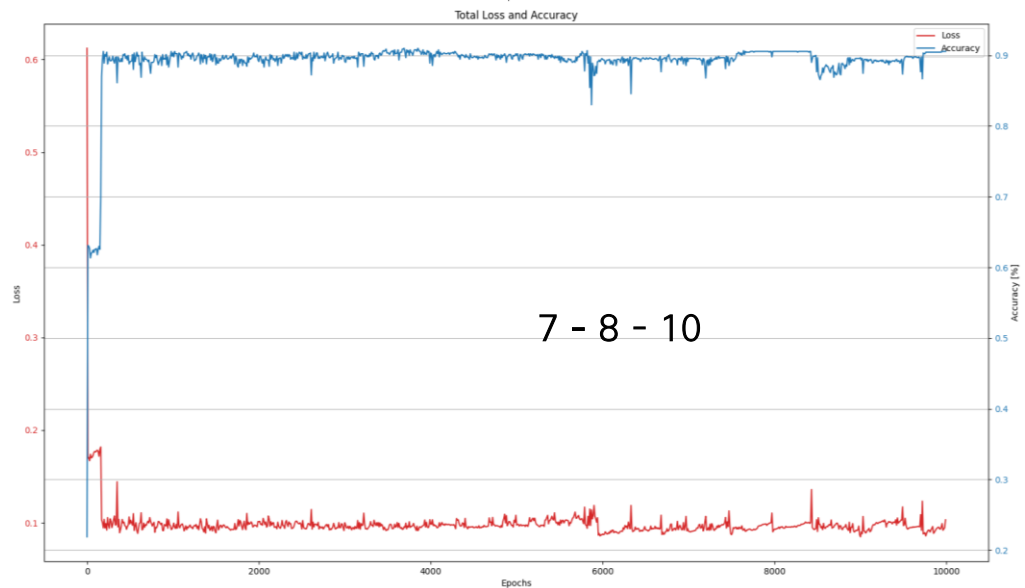
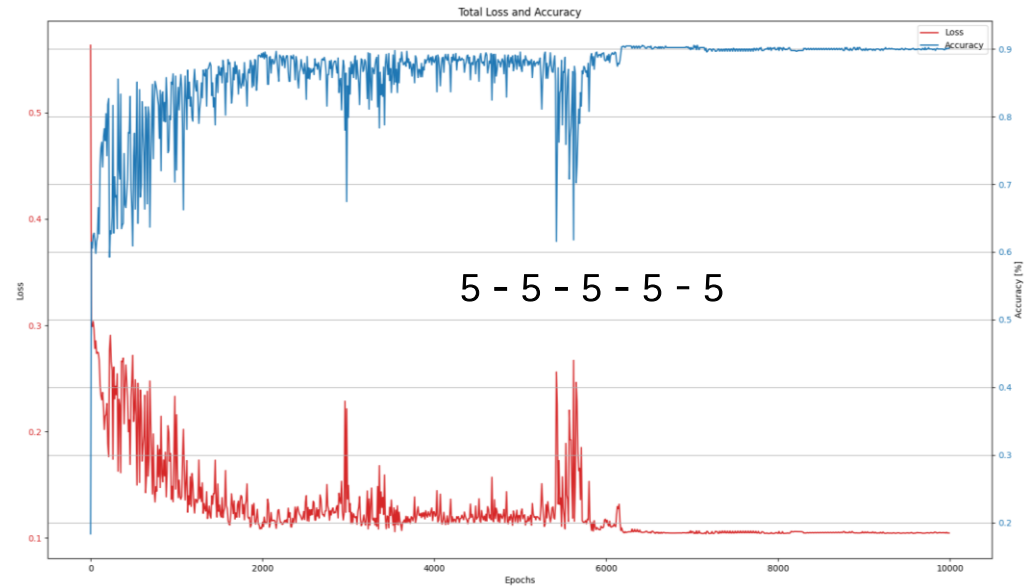
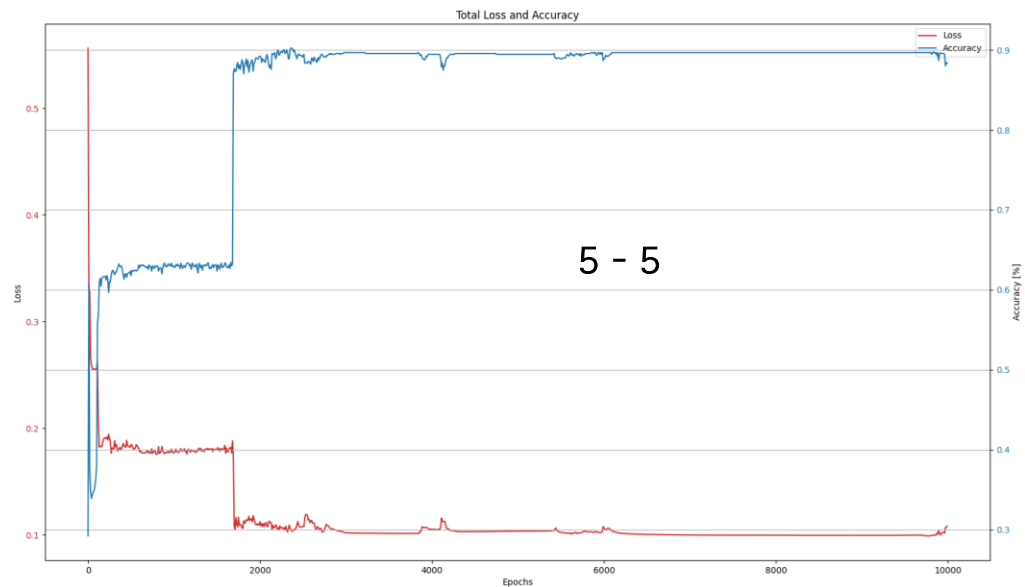
DNN (히든노드 총 15개, 5 - 5 - 5)

- 정확도: 90.44%
- 최종 오차: 0.098

DNN (히든노드 총 15개, 7 - 8 - 10)

- 정확도: 90.56%
- 최종 오차: 0.085

03 실습 #2 실행 결과



고급머신러닝 보고서
감사합니다